Stop Controlling Workflows with If Statements

State Machines in PHP

Joao Gilberto Magalhaes

byjg



About me

Brazilian, working out of Windsor, Ontario

20+ years writing software, most of it PHP

Built payment and wallet backends in production

Maintainer of `byjg/*` — including `byjg/statemachine` and Gluo, both used in today's demo

`opensource.byjg.com · github.com/byjg`



What you will leave with

1. How to **recognize** when your `if`s have become a workflow
2. How to **model** it — states, transitions, guards, actions
3. How to **build** one, step by step, then watch it run
4. Where the workflow **lives** in a real app
5. How to **test** it without a controller, a database or a clock

The talk is about the concepts.

The examples happen to use my library — so take the comparisons with that in mind. I have put my losses on a slide near the end.

Look at your schema

`status`	`situation`
`state`	`stage`
`workflow_step`	`current_step`

Every table with a column like this needs at least one of these:

Can it change?

What happens when it does?

So we answer them with an `if`

```
function handleGatewayWebhook(array $data): void
{
    if ($payment->status === 'pending') {
        $payment->status = 'paid';
        $payment->save();
        Mail::to($payment->user)->send(new PaymentConfirmed($payment));
    }
}
```

Nothing wrong with this line.

STEP 1 OF 3 — THE GATEWAY CALLS US

Every `if` is a bill

```php
function handleGatewayWebhook(array $data): void
{
    $payment = Payment::find($data['id']);
    if ($payment->status !== 'pending') {
        throw new InvalidState();
    }
    if ($data['status'] === 'succeeded') {
        approvePayment($payment);  // → paid
    }
}
```

3

paths to test

throw · succeeded → paid · other → nothing

The gateway says what happened. **We work out the new status.**

STEP 2 OF 3 — ADD ANALYSIS AND FAILED

Every `if` is a bill

```php
function handleGatewayWebhook(array $data): void
{
    $payment = Payment::find($data['id']);
    if ($payment->status !== 'pending') {
        throw new InvalidState();
    }
    if ($data['status'] === 'requires_review') {
        riskAnalysis($payment);    // → analysis
    } elseif ($data['status'] === 'declined') {
        failPayment($payment);     // → failed
    } else { // means succeeded
        approvePayment($payment);  // → paid
    }
}
```

4

paths to test

throw · review · declined · succeeded

The gateway drives the status now — **and one guard line decides what is allowed.**

STEP 3 OF 3 — THE SECOND WEBHOOK THREW

Every `if` is a bill

```php
function handleGatewayWebhook(array $data): void
{
    $payment = Payment::find($data['id']);
    if (!in_array($payment->status,
            ['pending', 'analysis'])) {
        throw new InvalidState();
    }
    if ($data['status'] === 'requires_review') {
        riskAnalysis($payment);    // → analysis
    } elseif ($data['status'] === 'declined') {
        failPayment($payment);     // → failed
    } else { // means succeeded
        approvePayment($payment);  // → paid
    }
}
```

7

paths to test

2 statuses × 3 results + throw

So we loosened the guard. **Now pending + succeeded skips analysis** — and any status we never heard of lands in that `else`.

THE CONSEQUENCE

Five tests, all green. The bugs are the other two.

STATUS	GATEWAY SAYS	REACHES	
`pending`	`requires_review`	`riskAnalysis()`	✓ tested
`pending`	`declined`	`failPayment()`	✓ tested
`analysis`	`succeeded`	`approvePayment()`	✓ tested
`analysis`	`declined`	`failPayment()`	✓ tested
`paid`	`succeeded`	`throws InvalidState`	✓ tested
`pending`	**`succeeded`**	`approvePayment()`	**skips analysis**
`analysis`	**`expired`**	`approvePayment()`	**unknown → paid**

Tests cover the paths you wrote. **The bill is for the ones you didn't.**

SAME RULES, MORE DOORS

Who else writes the status?

DOOR 1

Webhook

The gateway posts a result — the function you just saw.

DOOR 2

Admin button

The risk team approves a payment in analysis.

DOOR 3

Retry job

A call timed out, so the provider sends it again.

Three doors. Each one copied the ifs.

And they do not live together

<? StripeWebhookController.php

```php
if ($payment->status === 'pending') {
  $payment->status = 'paid';
  $payment->save();
  Mail::to($payment->user)->send(
    new PaymentConfirmed($payment));
}
```

! When **pending** → set to **paid** and send email

<? Admin/PaymentController.php

```php
// no check of current status

if ($gateway->charge($payment)) {
  $payment->status = 'paid';
}
```

! Charges and sets **paid** without checking the current status

<? Jobs/RetryPayment.php

```php
if ($payment->status === 'pending') {
  if ($gateway->charge($payment)) {
    $payment->status = 'paid';
  }
}
```

! When **pending** → charges and sets **paid**

! **Business rules are scattered across controllers, jobs and services.**
Hard to maintain, easy to break.

The bug that ships

10:05:00	retry job	Reads 'pending'. charge() → 'paid'. Receipt sent.
10:40:00	admin button	Old ticket: support clicks Charge. No status check — charged again.

Your customer is charged twice.

This is the moment to recognize

You have a workflow — not a column — the day any of these is true:

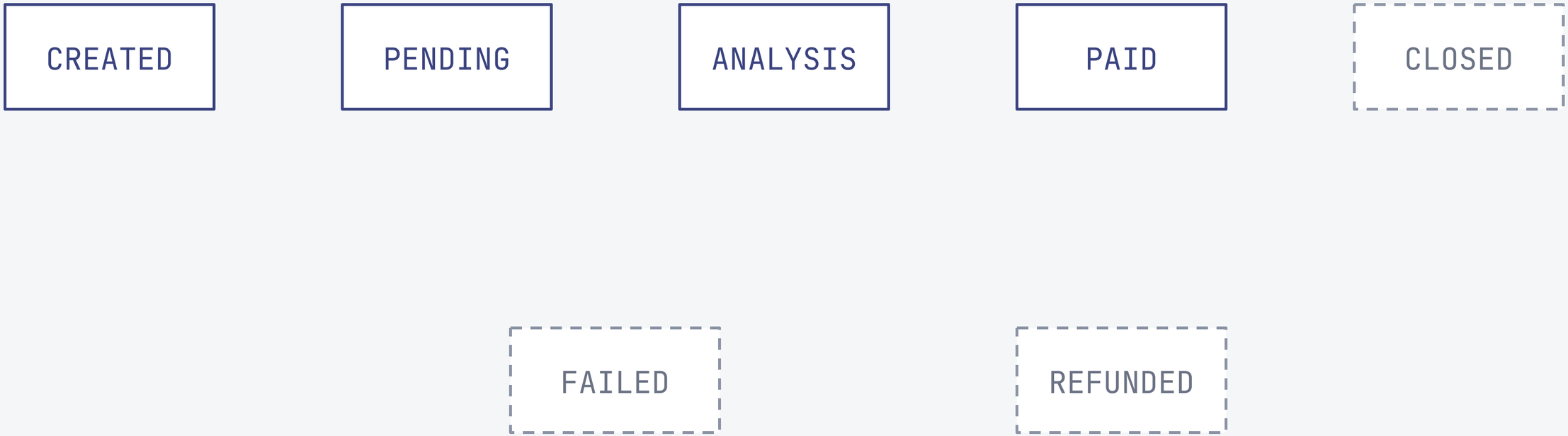
> The same status is written in **more than one file**

> A move is legal **from some states and not others**

> Something must **happen** when it moves — mail, stock, money

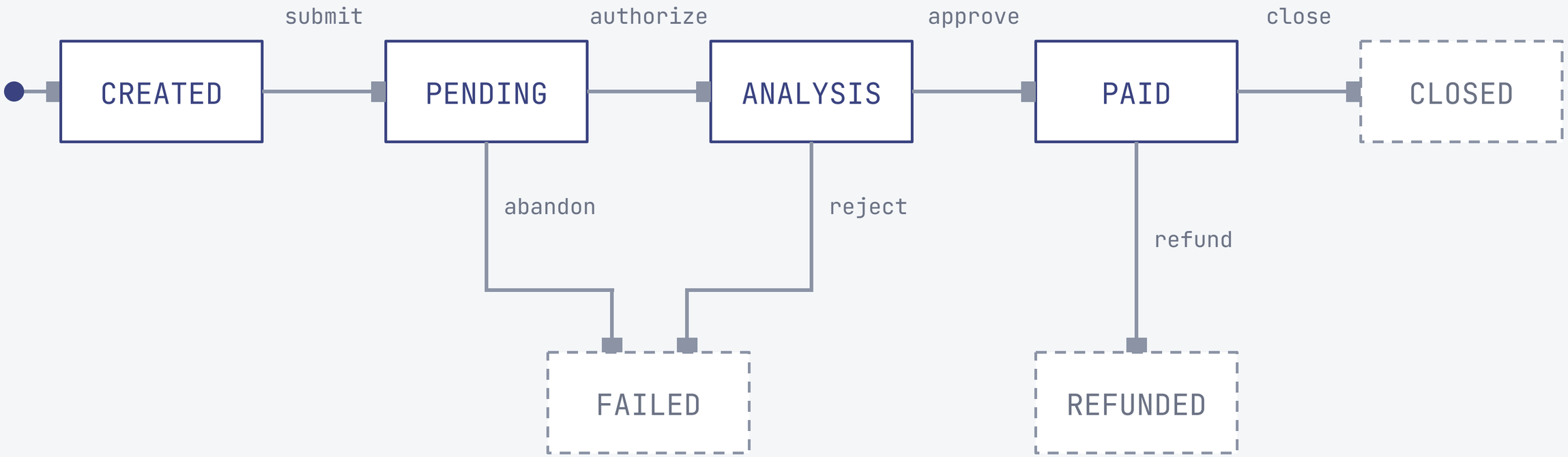
> Someone asked *“can it go from X to Y?”* and you had to **go and read code** to answer

What a state machine actually is



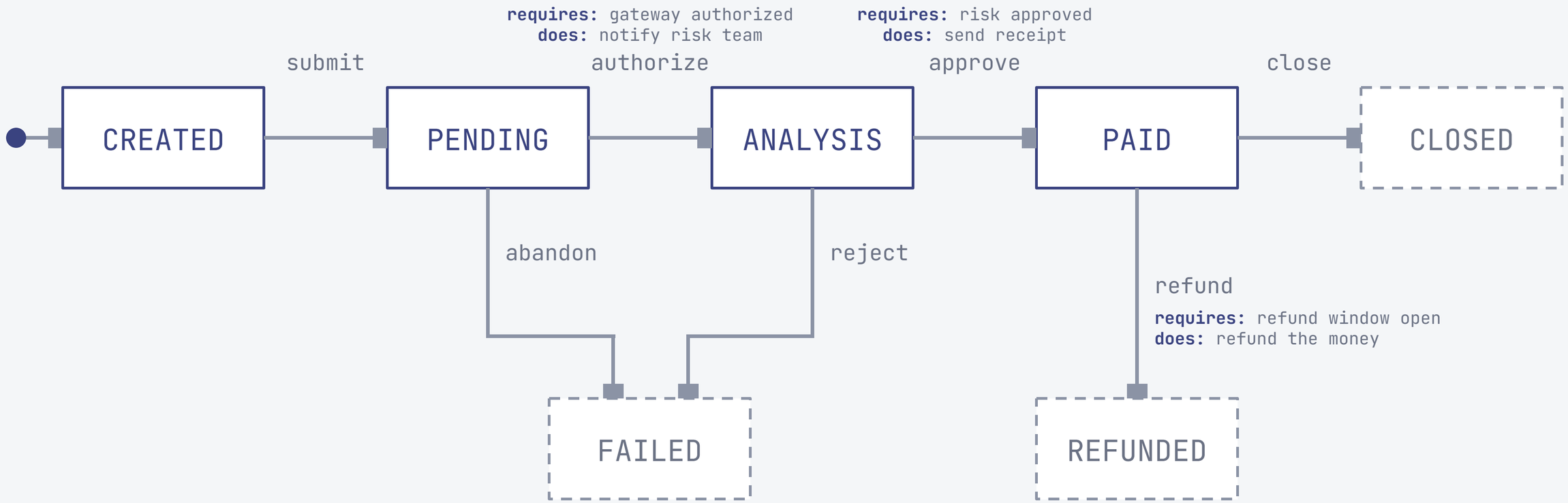
A closed set of **states**

What a state machine actually is



The **moves** allowed between them

What a state machine actually is



What each move **requires** and what it **does**

The same shape, four domains

	states	a guard	an action
Payment	`created → pending → analysis → paid`	risk approved	send receipt
Withdrawal	`requested → approved → sent`	under daily limit	call the bank
Onboarding	`invited → active → suspended`	accepted the terms	provision the account
Identity (KYC)	`pending → verified / rejected`	documents match	unlock the wallet

The rest of this talk is the first row. **All four are the same three things.**

The vocabulary, mapped to code

THE IDEA	THE CODE
a state	`a case of an enum`
a move	`Transition`
what it requires	`TransitionConditionInterface`
what it does	`TransitionActionInterface`

Four ideas. Four things to type. **That is the whole API.**

Before we build: the one rule

The machine **answers questions** . It never writes to your database, and it never fires a side effect on its own.

```
$next = $machine→transition($from, $to, $data);   // 1. may I?   → State, or null
$repo→save($payment, $next);                      // 2. commit
$next→process();                                  // 3. side effects
```

Check → move → then act.

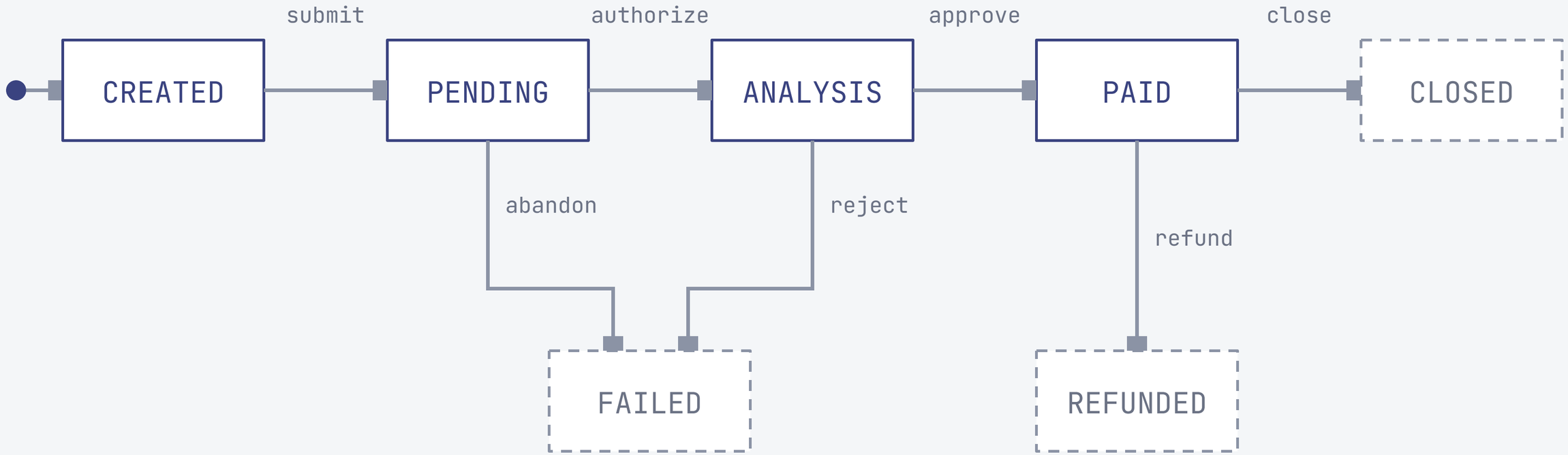
STEP BY STEP

Let’s build one

From an empty enum to a running machine.

```
created → pending → analysis → paid → closed
```

The graph we are about to build



Seven states, seven arrows — no arrow from `PENDING` straight to `PAID`, nothing out of `CLOSED`. **What is missing is the design.**

1. The states

```
enum PaymentState: string
{
    case Created  = 'CREATED';
    case Pending  = 'PENDING';
    case Analysis = 'ANALYSIS';
    case Paid     = 'PAID';
    case Failed   = 'FAILED';
    case Refunded = 'REFUNDED';
    case Closed   = 'CLOSED';
}
```

Not “the states we remembered”. **All of them** — and the compiler agrees.

2. The moves

```
$payments = FiniteStateMachine::createMachine(PaymentState::class)
    →addTransition(new Transition(PaymentState::Created,  PaymentState::Pending))
    →addTransition(new Transition(PaymentState::Pending,  PaymentState::Analysis))
    →addTransition(new Transition(PaymentState::Analysis, PaymentState::Paid))
    →addTransition(new Transition(PaymentState::Paid,     PaymentState::Refunded))
    →addTransition(new Transition(PaymentState::Paid,     PaymentState::Closed))
    →addTransitions(Transition::createMultiple(
        [PaymentState::Pending, PaymentState::Analysis], PaymentState::Failed
    ));
```

That is the diagram from two slides ago, **in a file**.

3. Where the `ifs` went

```php
final class RiskApproved implements TransitionConditionInterface
{
    public function canTransition(?array $data): bool
    {
        return ($data['gateway'] ?? null) === 'succeeded';
    }
}
```




```php
// slide 23's approve line, edited in place
→addTransition(new Transition(
    PaymentState::Analysis, PaymentState::Paid,
    new RiskApproved()
))
```

The rule now has **a name**, **one home** — and it guards **one arrow**.

Read it aloud: a payment in **`ANALYSIS`** may move to **`PAID`** only when **`RiskApproved`** says yes.

4. Where the emails went

```php
final class SendReceipt implements TransitionActionInterface
{
    public function __construct(private readonly Mailer $mailer) {}

    public function execute(State $from, State $to, ?array $data): void
    {
        $this->mailer->send(new PaymentConfirmed($data['payment']));
    }
}
```




```php
->addTransition(new Transition(
    PaymentState::Analysis, PaymentState::Paid,
    new RiskApproved(),     // may it move?
    new SendReceipt($mailer)    // what happens after
))
```

`execute()` gets **both ends** — so one action object serves many arrows.

Read it aloud: when a payment moves from **`ANALYSIS`** to **`PAID`**, **`SendReceipt`** runs — after you saved it.

All three call sites collapse

```php
$paid = $payments->transition($payment->status, PaymentState::Paid, $payload);

if ($paid === null) {
    return;                          // a second charge — not a legal move, nothing happens
}

$repo->save($payment, $paid);        // commit the move
$paid->process();                    // then the receipt goes out, exactly once
```

The replay is impossible now

A second charge asks for `PAID → PAID`.

That arrow does not exist.

There is nothing to process. You did not *remember* to handle it — you **cannot express it**.

So where does it live?

```php
final class PaymentMachine
{
    public static function build(ContainerInterface $c): FiniteStateMachine
    {
        return FiniteStateMachine::createMachine(PaymentState::class, [
            [PaymentState::Created,  PaymentState::Pending,  null,                      null],
            [PaymentState::Pending,  PaymentState::Analysis, PaymentAuthorized::class,  NotifyRiskTeam::class],
            [PaymentState::Pending,  PaymentState::Failed,   IsAbandoned::class,        null],
            [PaymentState::Analysis, PaymentState::Paid,     RiskApproved::class,       SendReceipt::class],
            [PaymentState::Analysis, PaymentState::Failed,   RiskRejected::class,       NotifyCustomer::class],
            [PaymentState::Paid,     PaymentState::Refunded, InRefundWindow::class,     RefundMoney::class],
            [PaymentState::Paid,     PaymentState::Closed,   RefundWindowExpired::class, null],
        ], [$c, 'get']);
    }
}
```

One file. The graph, the guards and the effects, in seven lines you can read to a product owner.

Or the whole graph as YAML

```yaml
enum: 'App\Payment\PaymentState'
transitions:
  - from: ANALYSIS
    to: PAID
    condition: 'App\Payment\RiskApproved'
    action: 'App\Payment\SendReceipt'
```

Reviewable by someone who does not read PHP. Checked when the file is **read** , not when the move is attempted.

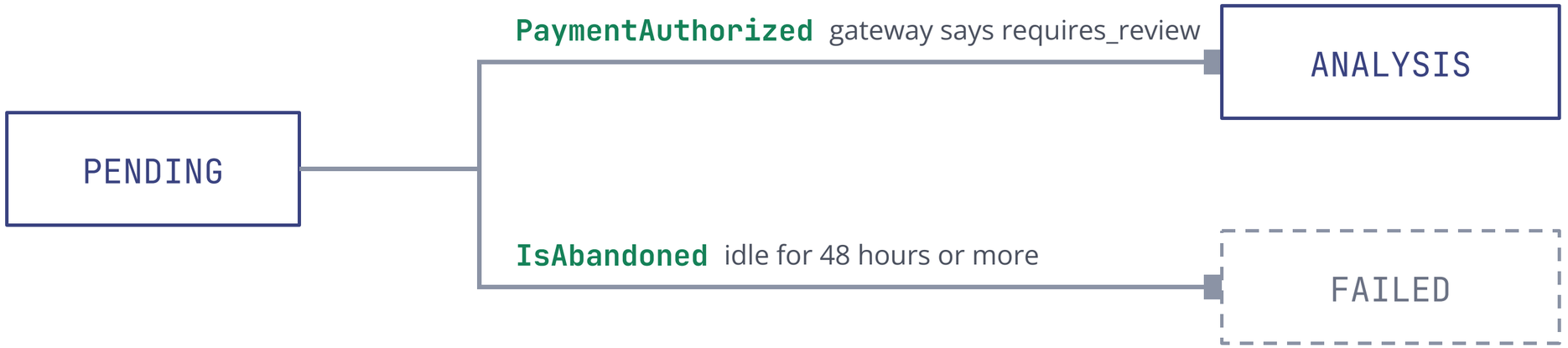
The five entry points

The webhook	The retry job	The admin screen	The refund endpoint	The nightly sweep

All five now ask the same object the same question.

THE WEBHOOK ONLY KNOWS WHAT HAPPENED

Sometimes the data decides



RiskApproved sits on ANALYSIS → PAID. From here it is **never asked**.

```
$fsm→autoTransitionFrom(PaymentState::Pending, ['gateway' ⇒ 'requires_review']); // ANALYSIS
$fsm→autoTransitionFrom(PaymentState::Pending, ['idle_hours' ⇒ 72]); // FAILED
$fsm→autoTransitionFrom(PaymentState::Pending, ['gateway' ⇒ 'succeeded']); // null — no arrow asks this
$fsm→autoTransitionFrom(PaymentState::Analysis, ['gateway' ⇒ 'expired']); // null — nobody says yes
```

It only asks the arrows leaving where you are. First yes wins; no yes, no move.

When two rules both say yes

```php
// slide 28's two ways out of ANALYSIS, now ranked: 6th element = priority
[PaymentState::Analysis, PaymentState::Paid,   RiskApproved::class, SendReceipt::class],
[PaymentState::Analysis, PaymentState::Failed, RiskRejected::class, NotifyCustomer::class, null, 100],

$machine->selectWith(new HighestPriority());   // most severe outcome wins
```

Sanctioned country **and** trusted customer? Declaration order should not decide that.

Persist first. Act second. Always.

```php
$next = $machine->autoTransitionFrom($current, $data);
if ($next === null) { return; }  // no legal move: nothing to save, nothing to run
$repo->save($entity, $next);   // your transaction, your ordering
$next->process();              // side effects only after the write survived
```

The machine stops at step 2. **Step 3 is yours to call.**

And the tests flip

BEFORE	AFTER
One test per `if`, per file	**One** table test over the graph
Rules tested 3×, wrong 3 ways	Each condition tested once, alone
Call sites full of branches	Call sites have no branches left

PHPUNIT

The graph is an executable spec

```php
public static function moves(): array
{
    return [
        [PaymentState::Analysis, PaymentState::Paid,     true ],
        [PaymentState::Paid,     PaymentState::Paid,     false],   // the second charge
        [PaymentState::Pending,  PaymentState::Paid,     false],   // skipping analysis
        [PaymentState::Closed,   PaymentState::Refunded, false],   // nothing leaves CLOSED
    ];
}

#[DataProvider('moves')]
public function testTheGraph(PaymentState $from, PaymentState $to, bool $exists): void
{
    $this->assertSame($exists, $this->fsm->getTransition($from, $to) !== null);
}
```

Your business rules, **as a PHPUnit data provider**.

PHPUNIT

...and the guards ride on it

```php
public static function decisions(): array
{
    return [
        [PaymentState::Analysis, PaymentState::Paid,     ['gateway' => 'succeeded'], true ],
        [PaymentState::Analysis, PaymentState::Paid,     ['gateway' => 'expired'],   false],  // nobody says yes
        [PaymentState::Paid,     PaymentState::Refunded, ['days_since_paid' => 3],   true ],
        [PaymentState::Paid,     PaymentState::Refunded, ['days_since_paid' => 45],  false],  // window is over
    ];
}

#[DataProvider('decisions')]
public function testTheRules(PaymentState $from, PaymentState $to, array $data, bool $allowed): void
{
    $this->assertSame($allowed, $this->fsm->canTransition($from, $to, $data));
}
```

Structure first, then the rules on top of it. Same table, one more column.

A condition, tested with nothing

```php
public function testAbandonedAfterFortyEightHours(): void
{
    $rule = new IsAbandoned(afterHours: 48);

    $this->assertTrue($rule->canTransition(['idle_hours' => 72]));
    $this->assertFalse($rule->canTransition(['idle_hours' => 12]));
}
```

No container. No database. No HTTP. No frozen clock.

What that leaves in your controller

```
public function handle(Request $request): Response
{
    $next = $this->payments->autoTransitionFrom($payment->status, $request->all());

    if ($next === null) { return response()->noContent(); }

    $this->repo->save($payment, $next);
    $next->process();

    return response()->noContent();
}
```

No branches left to test.

LIVE DEMO

Let’s see it run

Two apps, one state machine — clone it and run it yourself.

`github.com/byjg/longhorn-2026-code-support`



BEING HONEST

The others

Spatie, Symfony — and where this one fits.

We are not the first

THE STATE PATTERN

Spatie

`laravel-model-states`

The **State pattern** on Eloquent models. Each state is a class with its own behaviour, and each transition is a class that saves the model. Laravel only.

A PETRI NET ENGINE

Symfony Workflow

`symfony/workflow`

Petri nets *and* state machines — so one payment can be **settling and invoicing at the same time**. Seven lifecycle events, guard expressions, and `workflow:dump` draws your graph. Standalone-capable.

A TRANSITION TABLE

byjg/statemachine is neither of those. The states are a plain enum. The conditions and actions live outside them, on the transitions.

Also worth a look

ENUMS AND ATTRIBUTES

enum-state-machine

`llbbl/esm`

A state machine driven by an enum: transitions, guards and hooks are attributes on its cases. Framework-agnostic.

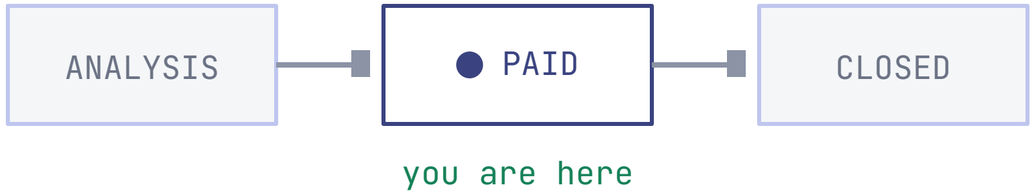
`github.com/llbbl/esm`

Wait — what is a Petri net?

A graph where the **tokens** are the state. The only question is how many you get.

STATE MACHINE

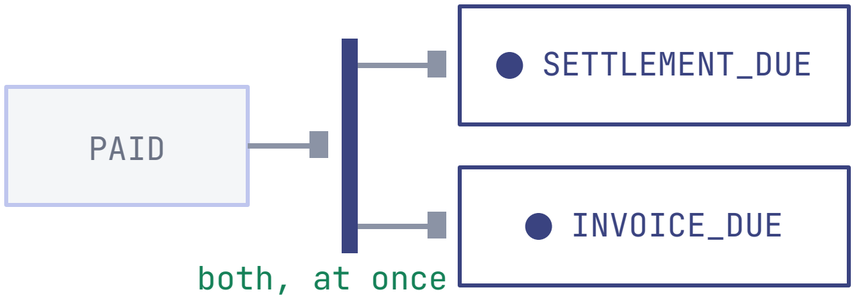
Exactly **one** token.



One token moves along the arrows. **A `status` column can hold this.**

PETRI NET

As **many** as you like.



Two tokens, two live places. **One column cannot hold this.**

THE SAME RULE, THREE LIBRARIES — 1 OF 3

byjg/statemachine

```
enum PaymentState: string {
    case Analysis = 'ANALYSIS';
    case Paid     = 'PAID';
}

$machine = FiniteStateMachine::createMachine(PaymentState::class, [
    [PaymentState::Analysis, PaymentState::Paid, RiskApproved::class, SendReceipt::class],
], [$container, 'get']);

$next = $machine->transition($payment->status, PaymentState::Paid, $payload);
if ($next === null) { return; }

$repo->save($payment, $next);   // you commit
$next->process();              // then the receipt goes out
```

Notice: the graph is one array, and the library never touches your database.

spatie/laravel-model-states

```php
abstract class PaymentState extends State {
    public static function config(): StateConfig {
        return parent::config()->default(Pending::class)
            ->allowTransition(Analysis::class, Paid::class, ApprovePayment::class);
    }
}

class ApprovePayment extends Transition {
    public function handle(): Payment {
        $this->payment->state = new Paid($this->payment);
        $this->payment->save();                        // the library writes
        Mail::to(...)->send(new PaymentConfirmed());   // inside the move
        return $this->payment;
    }
}

$payment->state->transitionTo(Paid::class);
```

Notice: the state is a class that can carry behaviour — and the move owns the write.

THE SAME RULE, THREE LIBRARIES — 3 OF 3

symfony/workflow

```yaml
# config/packages/workflow.yaml
framework:
  workflows:
    payment:
      type: state_machine
      supports: ['App\Entity\Payment']
      places: [analysis, paid]
      transitions:
        approve:
          from: analysis
          to: paid
```

```php
// the guard is a listener
#[AsEventListener('workflow.payment.guard.approve')]
public function guard(GuardEvent $e) {
    $e→setBlocked(!$this→approved($e));
}

if ($wf→can($payment, 'approve')) {
    $wf→apply($payment, 'approve');
    $em→flush();  // after the events
}
```

Notice: the graph is config — which is why `workflow:dump` can draw it, and this component cannot.

So why another one?

Not because it is more complete. **It is not.**

	Spatie	Symfony	this
Framework	Laravel only	standalone OK	**any**
States defined by	classes	strings or enums	**enum**
Multiple places at once	—	**yes**	—
Graph visualisation	—	**yes**	—
Event lifecycle	—	**7 events**	1 action
Owns persistence	yes	writes the marking	**no**
Destination from data	—	list, then pick	**one call + policy**

Because it is smaller — and it never sees your object.

The machine never sees your object

	SPATIE	SYMFONY	THIS
The machine never sees your object	Eloquent model + trait + state classes	a marking property it writes into	**nothing**
→ so it stays small runtime dependencies	Laravel	an event dispatcher	**none — just PHP**
→ so it drops into a stack you already assembled	no — Eloquent only	partial — runs standalone, brings its idiom	**yes**

Not more features. Less responsibility.

BYJG/GLUO-LARAVEL

On Laravel? There is a connector

The component needs no adapter to run under Laravel — the container is already a valid resolver. **The connector exists for what the four lines of wiring leave you.**

```php
class Payment extends Model implements StatefulModel
{
    use HasStateMachine;
    protected string $stateMachine = 'payment';
    protected $casts = ['status' ⇒ PaymentState::class];
}
// graph: config/gluo.php → statemachine.machines.payment
$payment→transitionTo(PaymentState::Paid, $data);
$payment→autoTransition($data);  // the data decides
```

WHEN THE ACTION RUNS

No transaction	**after the write**
In `DB::transaction()`	**after the commit**
Rolled back	**never**

Laravel ergonomics — a trait, a cast, a `CanTransitionTo` rule that answers 422 — **and the machine still owns nothing.**

```
composer require "byjg/gluo-laravel:7.0.x-dev" "byjg/statemachine:7.0.x-dev"
```

When not to reach for this

- One `if`, in one file, with nothing hanging off it → **leave it alone**
- Your stages are invented at **runtime**, per tenant → wrong tool, the enum is closed
- You need **several places at once** (approve *and* ship) → that is a Petri net, use Symfony
- You are deep in Symfony already → **use Symfony Workflow**, honestly

Take this home

1. Your `status` column already has rules. They are just scattered across `if`s.
2. Writing the graph down turns 30 branches into **one testable spec**.
3. Behaviour belongs to the **arrow**, not to the state.
4. **Persist first, act second.** Always.

```
composer require "byjg/statemachine:7.0.x-dev"
```

7.0 is the development branch — everything in this talk uses its API.

opensource.byjg.com

Thank you

Questions?



joind.in

`github.com/byjg/php-statemachine`

Slides + runnable code: github.com/byjg/longhorn-2026-code-support



NOT IN THE FLOW

Backup

“But Symfony has seven events.”	B1 · comparison	52
“What if two conditions both say yes?”	B2 · in depth	53
“Card or EFT — same state, different action?” (parity with Symfony)	B2 · in depth	54
“Is there a simpler example?”	B2 · in depth	55

BACKUP B1 · SYMFONY/WORKFLOW

“Symfony has 7 events. You have 1.”

Not seven actions — seven **moments** to hook. Yours has one.

Symfony event	Fires	Here
`guard`	before anything, can block	`TransitionConditionInterface`
`leave / transition / enter`	mid-move, marking not written	—
`entered / completed`	marking written	≈ the action
`announce`	once per newly enabled transition	—

Choosing the winner, in full

```php
$machine->selectWith(new HighestPriority(new RejectAmbiguous()));
```

`FirstDeclared` — first match wins (**default**; stops at the first match)

`RejectAmbiguous` — more than one match is an error, and it names them

`HighestPriority` — rank the outcomes; ties fall through to the tie-breaker

Named routes: same destination, different money

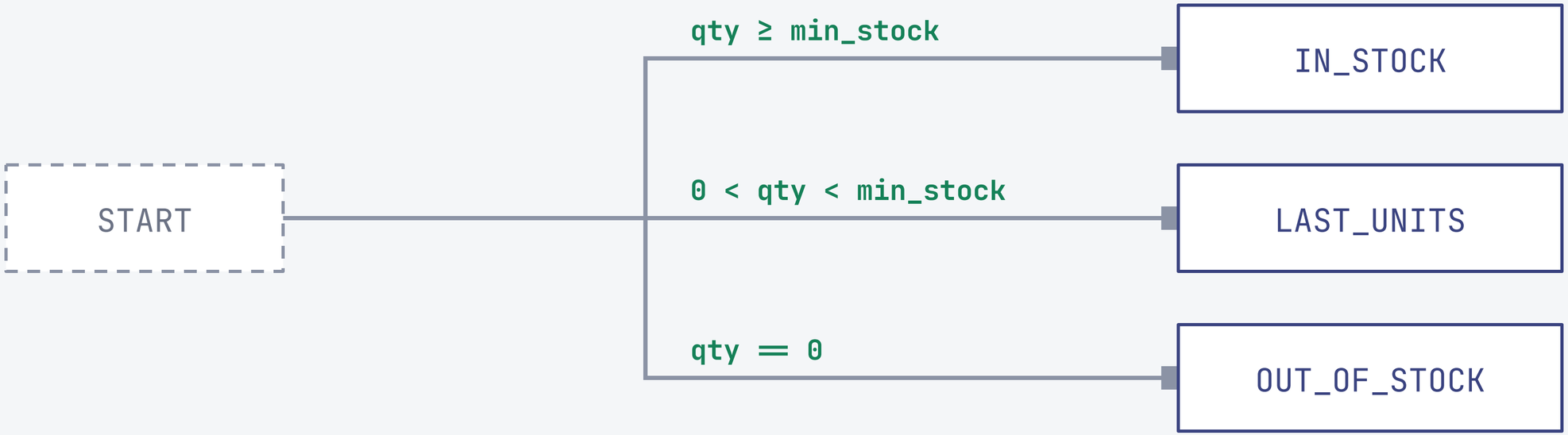



```php
->addTransition(Transition::named('CARD', Pending, Analysis, $authorizedByCard, $holdCard))
->addTransition(Transition::named('EFT',  Pending, Analysis, $debitedByEft,  $confirmEft))

$machine->transition(Pending, Analysis, $data);          // throws: two routes, name the one you mean
$machine->transition(Pending, Analysis, $data, 'CARD');  // ok
$machine->autoTransitionFrom(Pending, $data);            // ok: the data picks the route
```

The column alone has lost this: `ANALYSIS` does not say whether it was EFT or card. The named transition does.

The data decides: stock levels




```php
$machine->autoTransitionFrom(Stock::Start, ["qty" => 30, "min_stock" => 20]); // IN_STOCK
$machine->autoTransitionFrom(Stock::Start, ["qty" => 10, "min_stock" => 20]); // LAST_UNITS
$machine->autoTransitionFrom(Stock::Start, ["qty" =>  0, "min_stock" => 20]); // OUT_OF_STOCK
```